

COS30008 – Practice 1

Time: 2 hours

Total Marks: 396

Instructions: Implement all required methods according to specifications. Use provided test drivers to verify your implementation.

◆ Problem 1: Augmented Binary Search Tree Infrastructure (120 marks)

Implement an augmented BST that tracks **node count in each subtree** for efficient rank queries.

Required methods:

- `bool insert(const T& aKey);`
- `bool remove(const T& aKey);`
- `const T* find(const T& aKey) const;`
- `BinarySearchTree(const T& aKey);`
- `~BinarySearchTree();`
- `const T& operator*() const;`
- `bool empty() const;`
- `bool leaf() const;`
- `size_t size() const;`
- `size_t depth() const;`
- `size_t rank(const T& aKey) const; (New)`

Specifications:

- Each node maintains a `subtreeSize` field.
 - `rank(aKey)` returns the number of nodes with keys less than `aKey`.
 - All other operations behave as in the original exam but must update `subtreeSize` correctly.
-

◆ Problem 2: Copy Control with Metadata (64 marks)

Enhance copy control to include a `creationTimestamp` field (e.g., `std::string` or `std::chrono::time_point`).

Required methods:

- `BinarySearchTree(const BST& aOtherBST);`
- `BST& operator=(const BST& aOtherBST);`
- `BST* clone() const;`

Specifications:

- Copy constructor and assignment must copy `creationTimestamp`.
 - NIL trees must throw `domain_error` if copied.
 - `clone()` must return a deep copy with a new timestamp.
-

◆ **Problem 3: Move Semantics with Logging (72 marks)**

Add logging to move operations using a static `std::vector<std::string> moveLog`.

Required methods:

- `BinarySearchTree(T&& aKey);`
- `BinarySearchTree(BST&& aOtherBST);`
- `BST& operator=(BST&& aOtherBST);`

Specifications:

- Log each move operation with a message like "Moved node with key: 42".
 - Source BST becomes NIL after move.
 - Move assignment must clean up existing resources.
-

◆ **Problem 4: Reverse In-order Iterator (90 marks)**

Implement a **reverse in-order iterator** (right-root-left traversal).

Required iterator class structure:

```

template<typename T>
class BinarySearchTreeReverseIterator {
private:
    using BST = BinarySearchTree<T>;
    using BSTNode = BST*;
    using NodeStack = std::stack<const BSTNode>;
    const BST* fBST;
    NodeStack fStack;
    void pushRightPath(const BST* aNode);
public:
    BinarySearchTreeReverseIterator(const BST* aBST);
    const T& operator*() const;
    BinarySearchTreeReverseIterator& operator++();
    BinarySearchTreeReverseIterator operator++(int);
    bool operator==(const BinarySearchTreeReverseIterator& aOther) const;
    bool operator!=(const BinarySearchTreeReverseIterator& aOther) const;
};

```

Specifications:

- Add `rbegin()` and `rend()` methods to BST.
- Iterator must traverse in descending order.

◆ Problem 5: Theoretical Questions (50 marks)

Answer the following in 1–2 sentences.

- What are the trade-offs between AVL trees and Red-Black trees in terms of performance and implementation complexity?** [6]
- Explain the concept of RAII and how it helps in resource management in C++.** [4]
- Why is `std::unique_ptr` preferred over raw pointers in modern C++?** [4]
- How can we implement a thread-safe BST in C++?** [6]
- What is the difference between `explicit` and `implicit` constructors in C++?** [4]
- Explain the use of `std::enable_if` in template metaprogramming.** [4]
- What is the difference between `std::move` and `std::forward`? When should each be used?** [6]
- Describe the time complexity of rank queries in an augmented BST.** [4]
- What are the benefits of using `constexpr` in C++?** [4]
- How would you design a BST that supports undo/redo operations?** [6]